

Brouillon

Checkpoint Semaine 4

14 mars 2026

Implémentation et Optimisations Algorithmiques

L'implémentation d'un solveur basé sur la programmation dynamique pour le problème #SAT ou MaxSAT se heurte intrinsèquement au mur de la complexité exponentielle. Au cours de ce projet, les premières expérimentations naïves ont rapidement mis en évidence des limitations critiques : des temps d'exécution se chiffrant en minutes, la saturation de la mémoire vive, et des erreurs de segmentation (*Segmentation Faults*) dues à l'explosion combinatoire.

Ce chapitre documente la série de choix d'implémentation stratégiques et d'optimisations bas niveau qui ont permis de faire chuter le temps d'exécution de la Procédure 1 de plusieurs minutes (voire l'échec) à une fraction de milliseconde (moins de 0.001 seconde), prouvant ainsi expérimentalement la viabilité des théorèmes étudiés.

Représentation des données : Les Bitsets

Le problème initial (Naïf) : Dans la Procédure 1, l'algorithme doit manipuler, pour chaque nœud de l'arbre, des ensembles de sous-ensembles de clauses (les *PS-sets*). Une approche naïve consisterait à représenter un sous-ensemble de clauses par un tableau de booléens ou une liste d'entiers. Pour une formule de m clauses, chaque opération ensembliste (union, intersection, différence) nécessiterait une boucle parcourant les m éléments, générant un surcoût mémoire et CPU inacceptable.

La solution technique : Nous avons implémenté une structure de **Bitset** (vecteur de bits) où chaque clause est représentée par un unique bit au sein de mots mémoire de 64 bits (`uint64_t`).

Impact sur la complexité :

- **Spatiale :** L'empreinte mémoire d'un sous-ensemble est divisée par 64 (ou par 32 selon l'architecture). Une formule de 128 clauses ne pèse plus que 2 entiers de 64 bits.
- **Temporelle :** Les opérations ensemblistes utilisent les opérateurs bit-à-bit natifs du processeur (`&`, `|`, `~`). L'intersection de deux ensembles s'effectue avec un parallélisme matériel total, réduisant drastiquement les constantes cachées de la complexité temporelle.

Indexation globale : Le Trie Binaire

Le problème initial (Naïf) : La programmation dynamique génère d'innombrables masques de clauses à travers l'arbre. Beaucoup de ces masques sont identiques. Si l'on stocke chaque masque de manière isolée, la vérification de l'existence d'un masque à l'échelle globale requiert une comparaison linéaire en $\mathcal{O}(N)$ (où N est le nombre de masques déjà découverts).

La solution technique : Un *Trie Binaire* (Arbre préfixe) a été développé pour stocker l'intégralité des sous-ensembles uniques générés par l'algorithme. Chaque branche de ce Trie représente une clause (présente ou absente). Lorsqu'un masque atteint une feuille du Trie, on lui attribue un identifiant entier unique et continu $(0, 1, 2, \dots)$.

Impact sur la complexité : La recherche ou l'insertion d'un masque de m clauses s'effectue en temps $\mathcal{O}(m)$, indépendamment du nombre de masques N déjà stockés en mémoire. Cet identifiant unique est le socle qui a rendu possible l'optimisation en $\mathcal{O}(1)$ détaillée dans la section suivante.

Eradication du goulot d'étranglement : Tableau d'accès direct en $\mathcal{O}(1)$

Le problème initial (Naïf) : Lors de la fusion *Bottom-Up* (Procédure 1), un nœud interne effectue le produit cartésien des masques de son enfant gauche et de son enfant droit. Pour chaque nouveau masque généré, il fallait vérifier s'il n'avait pas *déjà* été ajouté à l'ensemble du nœud courant (boucle interne). Avec une ps-width k , le produit cartésien génère k^2 masques, menant à une complexité locale en $\mathcal{O}(k^3)$ par nœud. Pour $k = 50\,000$, cela représentait des milliers de milliards d'opérations.

La solution technique : Nous avons introduit un *Timestamp Array* ou table d'accès direct (`seen_array`). Sachant que chaque masque possède un ID unique fourni par le Trie, ce tableau utilise cet ID comme indice. La valeur stockée à cet indice est l'identifiant du nœud de l'arbre en cours de traitement.

Impact sur la complexité : La boucle de vérification disparaît. Savoir si un nœud a déjà vu un sous-ensemble se résout par une simple condition : `if (seen_array[ps_id] == node_id)`. La complexité du filtre des doublons chute de $\mathcal{O}(k)$ à $\mathcal{O}(1)$. La complexité locale de fusion du nœud passe ainsi de $\mathcal{O}(k^3)$ à $\mathcal{O}(k^2)$, faisant tomber le temps d'exécution de plusieurs minutes à quelques millisecondes sur de larges instances.

Robustesse mémoire : Redimensionnement dynamique encapsulé

Le problème initial (Naïf) : Face à l'imprévisibilité de l'explosion de la ps-width lors de l'utilisation d'arbres de décomposition non optimaux (arbres aléatoires), la taille maximale requise pour le `seen_array` était inconnue au moment de la compilation. Allouer une taille fixe entraînait inévitablement des débordements de tampon (*Buffer Overflow*) silencieux, résultant en des *Segmentation Faults* imprévisibles et ruinant la stabilité des benchmarks.

La solution technique : La responsabilité de la gestion de la mémoire du `seen_array` a été encapsulée directement dans la structure du `BinaryTrie`. Puisque le Trie est l'entité qui génère les IDs uniques, il est le seul à savoir quand un ID s'apprête à dépasser la capacité du tableau. Un mécanisme de redimensionnement dynamique (`realloc`) a été implémenté au moment exact de la création d'un nouvel identifiant global.

Impact sur la complexité : Sur le plan spatial, le programme n'alloue que la mémoire strictement nécessaire. La stabilité est garantie à 100 %. Le solveur peut désormais allouer de manière fluide des millions d'entiers en mémoire sans jamais crasher, permettant d'observer la complétion d'exécutions extrêmes.

Algorithmique des graphes : La *Linear Branch Decomposition* topologique

Le problème initial (Naïf) : Générer un arbre de décomposition de manière purement aléatoire a mené à des exécutions désastreuses (plus de 300 secondes pour seulement 100 clauses). En ignorant la structure de la formule, l'arbre aléatoire sépare arbitrairement des variables et leurs clauses, provoquant des "coupures" immenses. La taille des ensembles mémorisés (ps-width) explosait de manière exponentielle.

La solution technique : Nous avons implémenté une décomposition en arbre linéaire (en forme de "peigne" ou *caterpillar*). Pour que cette approche soit valide, nous avons introduit une analyse topologique préalable : une clause n'est rattachée au chemin central de l'arbre *que* lorsque la variable d'indice maximum qu'elle contient a été introduite.

Impact sur la complexité : Ce choix garantit qu'aucune clause n'est gardée dans la coupure (*cut*) plus longtemps que strictement nécessaire. Appliqué à une formule respectant une certaine localité, cet arbre a permis de faire chuter la ps-width de 1,8 million à seulement 13, et le temps d'exécution de 9.2 secondes à 0.0006 seconde.

Théorie vs Pratique : La pureté des *Interval Bigraphs*

Le problème initial (Naïf) : Du côté de la génération en Python, le générateur d'instances de "Type 3" (conçu pour avoir une ps-width bornée) répartissait les variables dans des blocs en utilisant un opérateur modulo (%). Cela créait involontairement un comportement circulaire (un cycle liant la fin de la formule à son début). Sur la décomposition linéaire en C, ce cycle forçait plusieurs clauses à rester actives de la première à la dernière étape de l'arbre, transformant une complexité théoriquement linéaire en complexité exponentielle.

La solution technique : L'opérateur modulo a été supprimé au profit d'une répartition par blocs contigus (ex : les variables 1, 2, 3 dans le bloc 0 ; 4, 5, 6 dans le bloc 1).

Impact sur la complexité : Cette correction supprime les longues arêtes topologiques. La structure générée correspond désormais strictement à la définition théorique d'un graphe d'intervalles bipartis (*Interval Bigraph*). C'est cette ultime correction théorique qui a validé la synergie parfaite entre l'instance, la décomposition et le solveur dynamique.

Fondements théoriques : Les instances générées

Afin de valider les propriétés algorithmiques de notre solveur, nous avons développé des instances spécifiques inspirées de l'article de recherche principal (Sæther et al.).

Les formules générées, appelées informellement "Type 3", sont des formules k-SAT structurées modélisant des **Graphes d'Intervalles Bipartis (Interval Bigraphs)**. Dans ce type d'instance, les variables et les clauses respectent une contrainte de forte localité spatiale (ou temporelle) : les variables sont partitionnées en blocs ordonnés séquentiellement, et une clause ne peut lier que des variables appartenant au même bloc ou à un bloc strictement adjacent.

Cette architecture garantit mathématiquement (selon les corollaires de la littérature) que la ps-width de la formule est strictement bornée par une constante, pour peu que l'on utilise la *Linear Branch Decomposition* adéquate. La génération rigoureuse de ce type précis de formules était indispensable pour démontrer empiriquement que notre solveur est capable d'échapper à la complexité en $\mathcal{O}(2^n)$ sur des instances présentant de telles propriétés topologiques.

0.1 Élagage Top-Down : La Procédure 2 et le filtrage des états

Le problème initial : La Procédure 1 (parcours Bottom-Up) génère l'ensemble $\mathcal{P}'(u)$ qui contient toutes les affectations locales valides. Cependant, beaucoup de ces affectations locales mènent à des "impasses" globales (elles sont incompatibles avec le reste de la formule). Conserver ces états invalides pour l'évaluation finale du problème #SAT ou MaxSAT entraînerait une explosion des calculs (multiplications et additions inutiles sur des branches mortes).

La solution technique : Nous avons implémenté la Procédure 2, un parcours *Top-Down* (de la racine vers les feuilles) qui calcule $\mathcal{P}''(u)$, le sous-ensemble strict de $\mathcal{P}'(u)$ contenant uniquement les états menant à une solution globale valide. Pour optimiser ce filtrage, nous avons utilisé un tableau booléen en $\mathcal{O}(1)$ (`valid_F`) indexé sur l'ID du masque parent fourni par le Trie. Ainsi, lors du produit cartésien des enfants, nous vérifions instantanément si l'union de leurs masques appartient à l'ensemble valide du parent, sans jamais réinsérer de doublons dans le Trie.

Impact sur la complexité : Cette approche permet une réduction drastique du nombre d'états à traiter pour l'étape finale. Par exemple, sur des instances de test, un nœud pouvant contenir plus de 750 000 états locaux dans $\mathcal{P}'(u)$ s'est vu réduit à un seul état utile dans $\mathcal{P}''(u)$. De plus, l'évaluation de la racine permet de statuer instantanément et avec certitude sur la satisfiabilité globale (SAT/UNSAT) de la formule avant même de redescendre l'arbre.

0.2 Architecture générique : Séparation Structure / Évaluation

Le concept architectural : Une des forces majeures de notre implémentation, fidèle à l'article de référence, est la séparation stricte entre l'exploration structurelle et l'évaluation numérique.

L'implémentation : Les Procédures 1 et 2 sont **totale**ment génériques et agnostiques du problème final à résoudre (SAT, #SAT ou MaxSAT). Elles ne manipulent que la topologie des clauses via des **Bitsets** pour extraire le "squelette" des chemins viables de la formule. Ce n'est qu'au stade de la **Procédure 3** que la différenciation s'opère : le squelette élagué $\mathcal{P}''(u)$ sera parcouru une dernière fois pour y appliquer une "charge utile" mathématique.

- Pour **#SAT**, cette charge utile sera un compteur (addition/multiplication) pour dénombrer les solutions.
- Pour **MaxSAT**, cette charge utile sera un score (addition/maximisation) pour évaluer le nombre de clauses satisfaites.

Impact : Ce choix de conception évite de calculer des scores ou des compteurs complexes pour des états qui s'avèreraient finalement être des impasses. La complexité arithmétique de la Procédure 3 est ainsi confinée au strict minimum des chemins viables, garantissant des temps d'exécution optimaux.